



**COMPOSABLE
SECURITY**

REPORT

Security consultation for Lido

Prepared by: Composable Security

Report ID: LDO-1d0b9440

Test time period: 2026-07-15 - 2026-07-15

Report date: 2026-07-15

Version: 1.0

Visit: composable-security.com

Contents

1. Security consultation summary (2026-07-15)	2
1.1 Subject of Consultation	2
1.2 Scope	2
1.3 Issues	2
1.4 Consultation Results	5
1.5 Deployments	5
1.6 Disclaimer	6

1. Security consultation summary (2026-07-15)

1.1. Subject of Consultation

The **Composable Security** team was engaged by **Lido** to perform a security review of a hot fix update of the Lido Oracle, including:

- fix for an over-ejection on exact withdrawal coverage - when the predicted balance of validators to exit exactly matched the withdrawal demand, the ejector requested one additional validator to exit,
- fix for incorrect sweep calculation for validators to eject,
- fix for indexing MetaRegistry groups,
- fix for non-deterministic tree construction.

1.2. Scope

The assessment focused on changes introduced in version **8.0.3**.

GitHub repository:

<https://github.com/lidofinance/lido-oracle>

CommitID: 1d0b9440fda2dd6d421a1756fa5604269f972a6d

1.3. Issues

1.3.1 Over-ejection on exact withdrawal coverage

The Lido Oracle gets the unfinalized stETH amount to know how many ETH will be required to fulfill the withdrawal queue.

In the next step it selects a validator to be exited and predicts the Execution Layer balance on the epoch when the validator is withdraw. If the sum of predicted balance and the validators withdrawable amount was greater than the required ETH amount, the loop stops because there is enough ETH to fulfill all withdrawal request.

```
if predictable_el_balance + gwei_to_wei(total_balance_to_eject_gwei) >
    to_withdraw_amount:
    break
```

When the sum is equal to the required amount the loop should stop but it does not and exits another validator.

The Fix

The fix changes the sign $>$ to $\geq$.

```
if predictable_el_balance + gwei_to_wei(total_balance_to_eject_gwei) >=
    to_withdraw_amount:
    break
```

It is worth noting that the `to_withdraw_amount` might not be exact amount that will be withdrawn from the queue - it can become less when the share rate goes down or the rewards get lower than predicted.

1.3.2 Incorrect sweep calculation for validators to eject

The sweep cycle prediction used in the withdrawal epoch estimation was calculated incorrectly, which could skew the number of validators selected for exit.

The `_get_predicted_el_balance` function double-counted the sweepable excess balance of validators that were recently requested to exit but haven't started their CL exit yet (`exit_epoch == FAR_FUTURE_EPOCH`).

Their balance was added twice, in two independent places:

- once via `going_to_withdraw_balance_gwei`, using `get_predictable_full_inbound_balance` (the validator's full, uncapped balance),
- again via `_get_withdrawable_lido_validators_balance`, which - since these validators are still "active" and not yet `is_fully_withdrawable` - treats them like any other active validator and adds their sweep excess (`get_predictable_inbound_sweep`, i.e. balance above `MAX_EFFECTIVE_BALANCE`).

The Fix

The fix is to change the `get_predictable_full_inbound_balance` function to `get_predictable_inbound_balance` in calculation of `going_to_withdraw_balance_gwei`.

```
going_to_withdraw_balance_gwei = Gwei(
    sum(
        map(
            get_predictable_inbound_balance,
            validators_going_to_exit,
        ),
        Gwei(0),
    )
)
```

This splits each validator's full balance B into two non-overlapping pieces instead of double-counting:

- `going_to_withdraw_balance_gwei` now claims only the capped portion ($B - E$),
- `_get_withdrawable_lido_validators_balance` (unchanged) still adds the sweep excess E .

Total: $(B - E) + E = B$, counted exactly once.

1.3.3 Invalid indexing in `get_all_groups`

The logic for fetching operator groups (function `get_all_groups`) from the MetaRegistry contract incorrectly handled group IDs as zero-based.

The Fix

The fix changed order from zero-based to one-based.

```
def get_all_groups(self, block_identifier: BlockIdentifier) -> list[
    OperatorGroup]:
    # Group IDs are one-based: the contract reserves NO_GROUP_ID = 0
    # and creates real groups with IDs 1..groupsCount.
    all_groups = self.get_operator_groups_count(block_identifier)
    return [self.get_operator_group(group_id, block_identifier) for group_id in
            range(1, all_groups + 1)]
```

1.3.4 Non-deterministic tree construction

Lido Oracle build trees for rewards and strikes. The values used to build the tree are passed to the Merkle library in insertion order.

The library sorts leaf hashes when calculating the root, but preserves input order in the serialized values array.

Consequently, identical logical distributions can have the same Merkle root but different artifact bytes, IPFS CIDs, and report hashes.

The Fix

The fix additionally sorts the values passed to the Rewards and Strikes trees.

```
class RewardsTree(Tree[RewardsTreeLeaf]):
    decoder = RewardsTreeJSONDecoder
```

```

@classmethod
def new(cls, values) -> Self:
    """Create new instance around the wrapped tree out of the given values"""
    # Sort leaves by node operator id
    sorted_values = sorted(values, key=lambda leaf: leaf[0])
    return cls(StandardMerkleTree(sorted_values, ("uint256", "uint256")))

class StrikesTree(Tree[StrikesTreeLeaf]):
    encoder = StrikesTreeJSONEncoder
    decoder = StrikesTreeJSONDecoder

    @classmethod
    def new(cls, values) -> Self:
        """Create new instance around the wrapped tree out of the given values"""
        # Sort leaves by (node operator id, pubkey)
        sorted_values = sorted(values, key=lambda leaf: (leaf[0], leaf[1]))
        return cls(StandardMerkleTree(sorted_values, ("uint256", "bytes", "uint256[]")))

```

1.4. Consultation Results

After consultation, the implemented fixes effectively address the described issues.

For the **Non-deterministic tree construction** issue, the order of values returned by `items()` is deterministic (for Python 3.7+) – it preserves the insertion order.

For Oracle Lido, the oracle members use the same data source and the same oracle implementation, ensuring consistent order. However, the fix improves oracle security when using multiple data sources and may return the same data in different orders.

For the **Incorrect sweep calculation for validators to eject** issue, the fix reduces `going_to_withdraw_balance_gwei` and increases the precision of `withdrawal_epoch`.

However, the `get_predictable_inbound_balance` function should return the effective balance (not the total balance) so that the withdrawable amount is the same as what the CL client uses to calculate the exact withdrawal epoch.

1.5. Deployments

After the security review conducted, we verified that the Dockerfile used to build an image uses the reviewed source code. The published image with the manifest digest **sha256:0f97715ddd74e289b5133c1394984902a0027cfd0a91c2a85e1d9e741388dd4d** corresponds to the commitID: **1d0b9440fda2dd6d421a1756fa5604269f972a6d**.

1.6. Disclaimer

Security consultation **IS NOT A SECURITY WARRANTY.**

During the review, the Composable Security team makes every effort to detect any occurring problems and help to address them. However, it is not allowed to treat the report as a security certificate and assume that the project does not contain any vulnerabilities. Securing applications is a multi-stage process.

Therefore, we encourage the implementation of security mechanisms at all stages of development and maintenance.



Damian Rusinek

Smart Contracts Auditor

@drdr_zz

damian.rusinek@composable-security.com



Paweł Kuryłowicz

Smart Contracts Auditor

@wh01s7

pawel.kurylowicz@composable-security.com

